
# Chapter 12 and 17: Exception Handling with I/O

# Phase-I:
# Exception Handling

# Chapter 12: Exception Handling

# 12.1. Introduction to Exception Handling

# 👉Motivation

✨Exceptions are unexpected events that disrupt the normal flow of a program.

**Example**: Division by Zero Error

```
int result = 10 / 0; // This will throw an ArithmeticException
```

**Example**: Overflow Array Index

```
int[] arr = new int[3];
arr[5] = 10; // This will throw an ArrayIndexOutOfBoundsException
```

**Example**: Invalid Input

```
Scanner scanner = new Scanner(System.in);
System.out.print("Enter a number: ");
int number = scanner.nextInt(); // If input is not an integer, it will throw InputMismatchException
```

**Example**: File Not Found

```
FileReader file = new FileReader("nonexistent.txt"); // This will throw FileNotFoundException
```

**Example**: Null Pointer Access

```
String str = null;
System.out.println(str.length()); // This will throw NullPointerException
```

**Example**: Network Connection Error

```
Socket socket = new Socket("invalid.host", 8080); // This will throw UnknownHostException
```

**Example**: Database Connection Error

```
Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/my
db", "user", "password"); // This may throw SQLException if connection fails
```

**Example**: Parsing Error

```
String number = "abc";
int parsedNumber = Integer.parseInt(number); // This will throw NumberFormatException
```

**Example**: Out of Memory Error

```
List<Integer> largeList = new ArrayList<>();
for (int i = 0; i < Integer.MAX_VALUE; i++) {
    largeList.add(i); // This may throw OutOfMemoryError if memory limit is exceeded
}
```

**Example**: Stack Overflow Error

```
public void recursiveMethod() {
    recursiveMethod(); // This will eventually throw StackOverflowError
}
```

**Many more** ...

# 12.2. Exception Handling Overview

# 👉What is an Exception Handling?

✨The **Exception Handling** is a mechanism to handle runtime errors in a controlled manner.

✨It allows the program to continue execution or terminate gracefully instead of crashing.

✨It provides a way to catch and respond to errors, ensuring that the program can handle unexpected situations without losing data or functionality.

✨Exception handling is a key feature of Java that helps developers write robust and error-tolerant code.

## 👉Why Do We Need to Handle Exceptions?

✨To prevent program crashes and unexpected behavior.

✨To provide meaningful error messages to users.

✨To allow the program to recover from errors and continue execution.

**Example**: Unhandled Exception

```
int[] arr = {1, 2, 3};
System.out.println(arr[5]);  // Throws ArrayIndexOutOfBoundsException
```

**Output**:

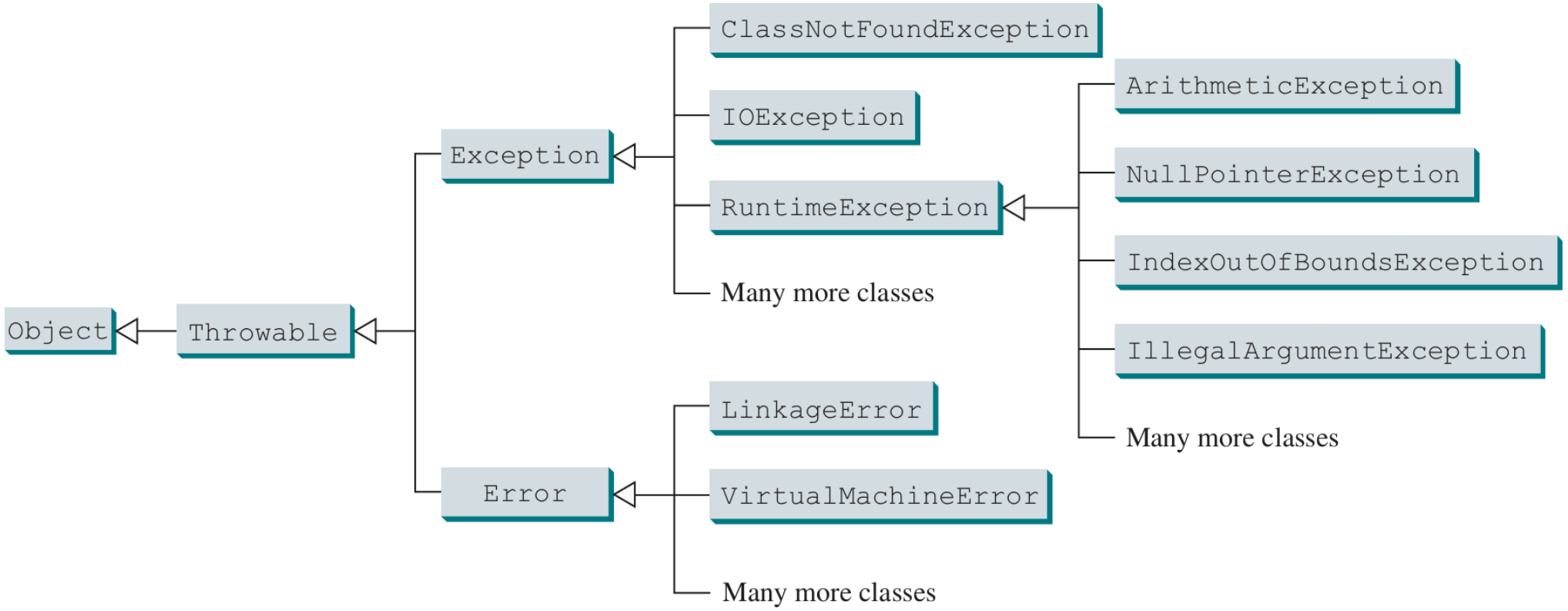
```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3 at Main.main(Main.java:5)
```

**Explain**: The program crashes with an unhandled exception, providing no opportunity for recovery or user-friendly error messages.

# 12.3. Exception Handling Type

# 👉Exception Handling Hierarchy

✨Exception Handling thrown are instances of the classes shown in this diagram, or of subclasses of one of these classes.



| Exception/Error | Reason |
| --- | --- |
| **Throwable** | Provides a common root for all errors and exceptions, enabling unified handling and reporting. |
| **Error** | Separates serious system-level failures from application-level issues, so programs don't try to recover. |
| **LinkageError** | Alerts developers to class loading or compatibility problems that require fixing the build or classpath. |
| **VirtualMachineError** | Notifies when the JVM is critically low on resources, signaling that the application cannot continue. |
| **OutOfMemoryError** | Warns when memory is exhausted, helping diagnose memory leaks or excessive resource usage. |
| **Exception** | Allows programs to catch and recover from expected but abnormal conditions, improving robustness. |
| **ClassNotFoundException** | Helps identify missing dependencies at runtime, so developers can resolve classpath issues. |

| Exception/Error | Reason |
| --- | --- |
| **IOException** | Enables handling of I/O failures, so programs can respond to file or network problems gracefully. |
| **RuntimeException** | Flags programming errors that should be fixed in code, not just handled at runtime. |
| **ArithmeticException** | Prevents undefined arithmetic operations from silently producing wrong results. |
| **NullPointerException** | Detects attempts to use uninitialized objects, helping catch bugs early. |
| **IndexOutOfBoundsException** | Ensures safe access to arrays and collections, preventing data corruption or crashes. |
| **IllegalArgumentException** | Encourages validation of method arguments, making APIs safer and more predictable. |

# 12.4. Declaring, Throwing, and Catching Exceptions

# 👉Declaring Exception Handling

✨When a method can throw a checked exception, it must declare it in its signature using the `throws` keyword.

✨This informs the caller that it must handle or declare the exception.

**Syntax**: Declaring Exceptions

```
public void methodName() throws ExceptionType1, ExceptionType2, ... {
    // Method implementation
}
```

# 👉Throwing Exception Handling

✨The `throw` statement is used to explicitly throw an exception.

✨It can be used to throw both checked and unchecked exceptions.

**Syntax**: Throwing Exceptions

```
throw new ExceptionType("Error message");
```

# 👉Catching and Printing Exception Info

✨The `catch` block is used to handle exceptions thrown by the `try` block.

✨It can catch specific exceptions or a general `Exception`.

✨The `printStackTrace()` method can be used to print the stack trace of the exception for debugging purposes.

**Syntax**: Catching Exceptions

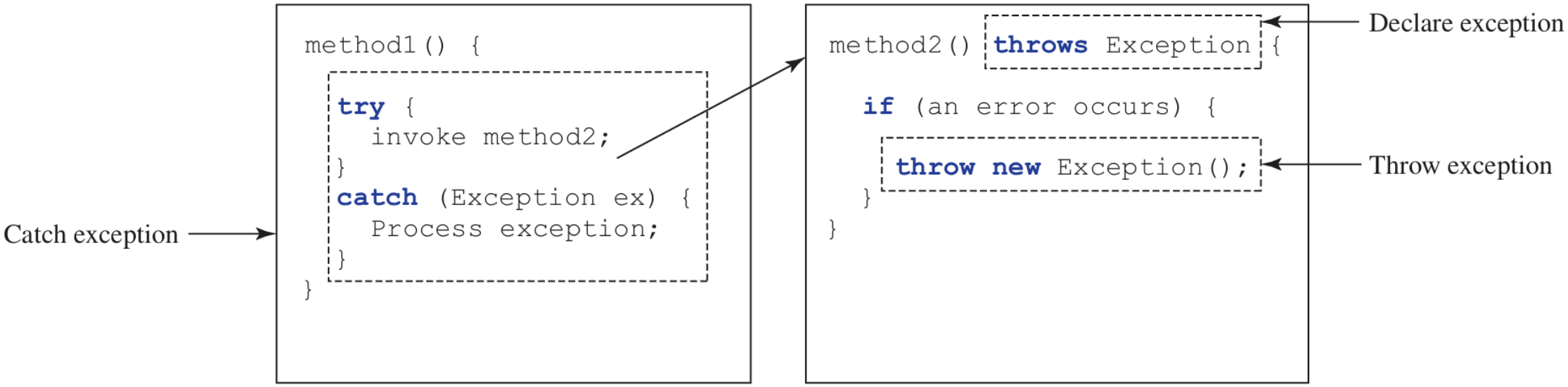
```
try {
    // Code that may throw an exception
} catch (ExceptionType ex) {
    ex.printStackTrace();  // Print stack trace for debugging
}
```

# 👉Throwing Exceptions in Class and Method

**Example**: Throwing Exceptions in Methods

```
public class ExceptionExample {
    public static void main(String[] args) {
        try {
            method1();
        } catch (Exception e) {
            e.printStackTrace(); // Print stack trace for debugging
        }
    }
    public static void method1() throws Exception {
        method2(); // This may throw an exception
    }
    public static void method2() throws Exception {
        throw new Exception("An error occurred in method2");
    }
}
```

# 👉Visualizing of Exception Handling in Chain Methods




**Explain**:

✨ `method1()` calls `method2()`, which may throw an exception.

✨ If `method2()` throws an exception, it propagates up to `method1()`.

# 👉Getting Information from Exceptions

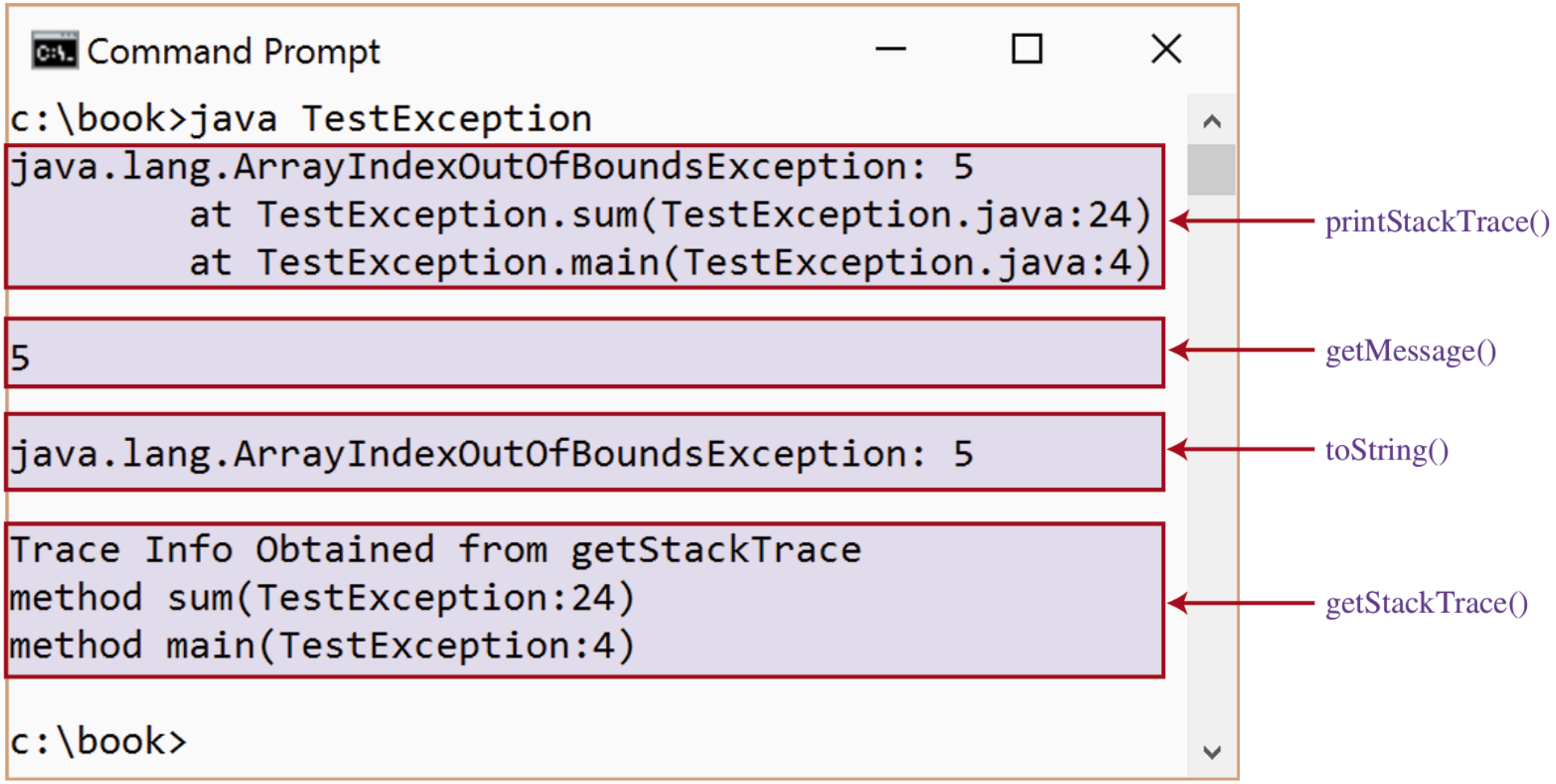
✨Exceptions provide methods to retrieve information about the error.

✨Common methods include:

- ✨ `printStackTrace()` : Prints the stack trace to the console for debugging.
- ✨ `getMessage()` : Returns a detailed message about the exception.
- ✨ `getCause()` : Returns the cause of the exception (if any).

## **Example**: Getting Information from Exceptions

```
public class TestException {
    public static void main(String[] args) {
        try {
            System.out.println(sum(new int[]{1, 2, 3, 4, 5}));
        } catch (Exception ex) {
            ex.printStackTrace();
            System.out.println("\n" + ex.getMessage());
            System.out.println("\n" + ex);
            System.out.println("\nTrace Info from getStackTrace:");
            for (StackTraceElement e : ex.getStackTrace())
                System.out.println("method " + e.getMethodName() + "("
                + e.getClassName() + ":" + e.getLineNumber() + ")");
    }}
    private static int sum(int[] list) {
        int result = 0;
        for (int i = 0; i <= list.length; i++)
            result += list[i];
        return result;
    }}                                                              .
```


Command Prompt
c:\book>java TestException
java.lang.ArrayIndexOutOfBoundsException: 5
at TestException.sum(TestException.java:24)
at TestException.main(TestException.java:4)
5
java.lang.ArrayIndexOutOfBoundsException: 5
Trace Info Obtained from getStackTrace
method sum(TestException:24)
method main(TestException:4)
c:\book>
printStackTrace()
getMessage()
toString()
getStackTrace()

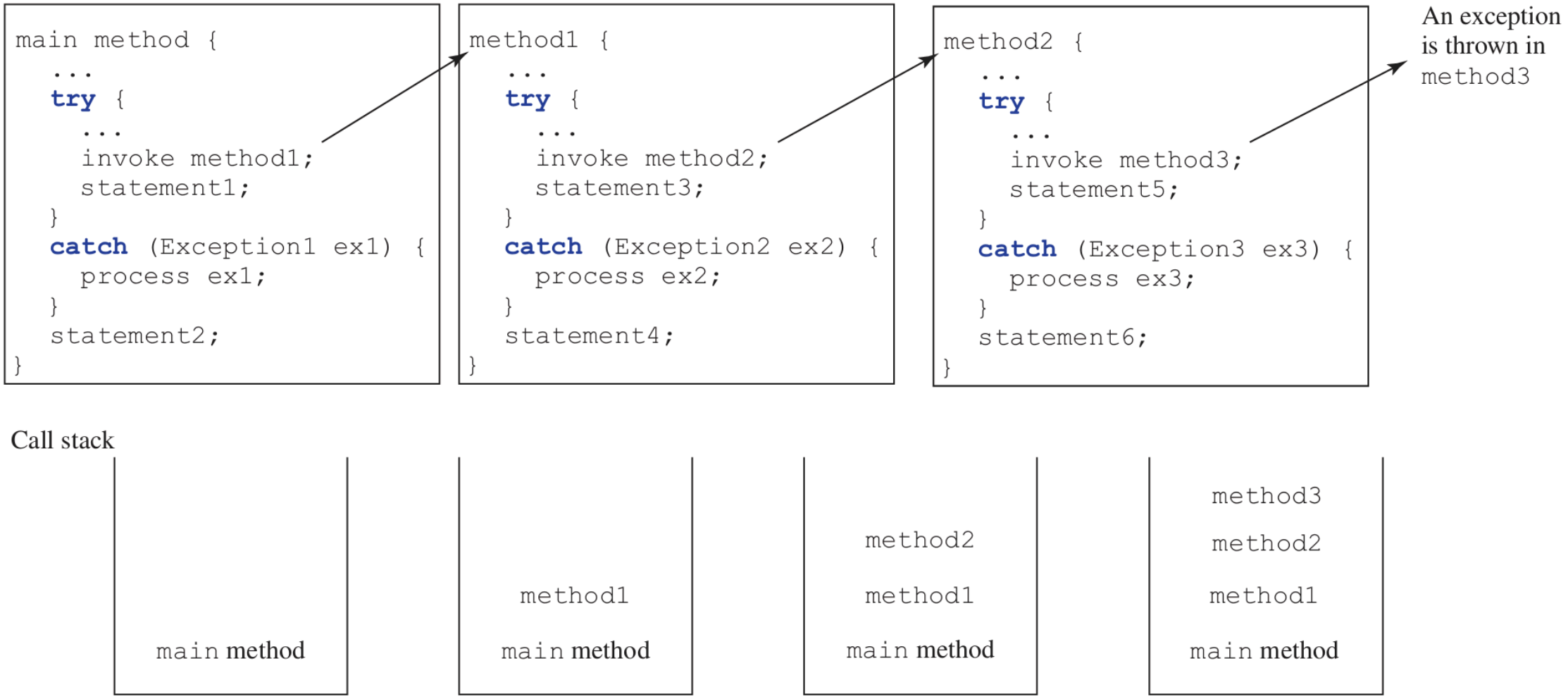
# 👉Call Stack Exception Handling

✨The **Call Stack** is a stack data structure that keeps track of method calls in a program.

✨The **Stack Trace** is a list of method calls that shows the sequence of method invocations leading to an exception.

✨This helps in debugging by identifying the source of the error.

# **Example**: Call Stack Exception Handling



**Explain**:

- ✨ When `main method` invokes `method1()`, it may throw an exception `ex1` and it keeps propagating up the call stack.
- ✨ `method1()` invokes `method2()`, which may throw another exception `ex2`.
- ✨ `method2()` invokes `method3()`, which may throw yet another exception `ex3`.
- ✨ The exception `ex3` is caught in the `catch` block of `method2()`, and its stack trace is printed.

# 👉Correct and Incorrect Order of Catch Blocks

✨Correct order of catch blocks:

```
try {
    // Code that may throw exceptions
} catch (SpecificException ex) {
    // Handle specific exception
} catch (GeneralException ex) {
    // Handle general exception
}
```

**Explain**:

✨Always catch specific exceptions before general ones to avoid errors.

✨This lets you handle each error type more precisely.

✨ Incorrect order of catch blocks (will cause a compile-time error):

```
try {
    // Code that may throw exceptions
} catch (GeneralException ex) { // This should not be before SpecificException
    // Handle general exception
} catch (SpecificException ex) {
    // Handle specific exception
}
```

**Explain**:

✨ If you catch a general exception before a specific one, the specific catch block becomes unreachable and causes a compile-time error.

✨ Always catch specific exceptions first.

# 12.5. The `Finally` Clause

## 👉What is the `finally` Clause?

- ✨The `finally` block is used to execute code that must run regardless of whether an exception occurs or not.
- ✨It is typically used for cleanup operations, such as closing files or releasing resources.
- ✨The `finally` block always executes after the `try` and `catch` blocks, even if an exception is thrown or caught.

**Syntax**: Using `finally`

```
try {
    // Code that may throw an exception
} catch (ExceptionType ex) {
    // Code to handle the exception
} finally {
    // Code that always executes
}
```

## **Example**: Using `finally` for Cleanup

```
FileReader file = null;
try {
    file = new FileReader("test.txt");
    // Read file
} catch (IOException ex) {
    System.out.println("Error reading file!");
} finally {
    if (file != null) {
        file.close();  // Ensures file is closed
    }
}
```

**Explain**:

- ✨ The `try` block attempts to read a file.
- ✨ If an `IOException` occurs, it is caught in the `catch` block.
- ✨ The `finally` block ensures that the file is closed, regardless of whether an exception occurred or not.

# 12.6. When to Use Exceptions?

# 👉Best Practices for Using Exceptions

✨Exceptional conditions that are not part of the normal flow of the program.

✨Situations where the program cannot continue without handling the error.

✨Unexpected conditions that require special handling.

✨Resource management (e.g., closing files, releasing connections).

✨Error handling in APIs or libraries where the caller needs to know about potential issues.

✨Logging errors for debugging purposes.

✨Providing meaningful error messages to users or developers.

# 👉Avoid Using Exceptions

✨For normal control flow or expected conditions.

✨For performance-critical sections where exceptions may be thrown frequently.

✨For simple checks that can be handled with conditional statements (e.g., checking for null values).

✨For cases where the program can continue without handling the error.

✨For cases where the error can be handled with a simple return value or flag.

✨For cases where the error can be anticipated and handled without throwing an exception.

**Example**: Avoid Overusing Exceptions

```
// Bad: Using exception for normal logic
try {
    System.out.println(refVar.toString());
} catch (NullPointerException ex) {
    System.out.println("refVar is null");
}

// Better: Use if-else
if (refVar != null) {
    System.out.println(refVar.toString());
} else {
    System.out.println("refVar is null");
}
```

**Explain**: Using exceptions for normal control flow is inefficient and can lead to performance issues. Instead, use conditional statements to handle expected conditions.

# 12.7. Rethrowing Exceptions

# 👉What is Rethrowing an Exception?

✨The **Rethrowing an Exception** is a technique where an exception caught in a `catch` block is thrown again.

✨It useful when a method cannot fully handle the exception or needs to add context.

**Syntax**: Rethrowing Exceptions

```
try {
    // Code that may throw an exception
} catch (IOException ex) {
    // Optional: log or perform some action
    throw ex; // Rethrow the exception
}
```

# 👉Best Practice for Rethrowing Exceptions

**1. Preserve the Original Stack Trace**: Basic rethrow (preserves stack trace)

```
try {
    // code that may throw
} catch (IOException e) {
    // perform cleanup if needed
    throw e;  // rethrow as-is
}
```

**2. Add Context with Exception Chaining**: Wrap with additional context

```
try {
    // code that may throw
} catch (IOException e) {
    throw new ServiceException("Failed to process request", e);  // cause is set
}
```

### 3. Use `final` for Checked Exceptions in Lambda Expressions:

```
public void processFiles(List<Path> files) throws IOException {
    // This variable will hold the first IOException encountered.
    // Any additional IOExceptions will be added as suppressed exceptions.
    IOException ex = null;
    for (Path file : files) {
        try {
            process(file); // Attempt to process each file.
        } catch (IOException e) {
            // If this is the first exception, store it.
            // Otherwise, add subsequent exceptions as suppressed to the first.
            if (ex == null) ex = e; else ex.addSuppressed(e);
        }
    }
    // After processing all files, if any exception occurred, throw it.
    // This preserves all exceptions that happened, not just the first.
    if (ex != null) throw ex;
}
```

**4. Rethrow with Additional Context (Java 7+)**: Using `addSuppressed` for cleanup errors.

```
try {
    // code that may throw
} catch (IOException original) {
    try {
        // cleanup that might also throw
    } catch (Exception cleanup) {
        original.addSuppressed(cleanup);
    }
    throw original;
}
```

### 5. **Rethrow Specific Exception Types**: Filter and transform exceptions

```
try {
    // code that may throw multiple exceptions
} catch (FileNotFoundException e) {
    throw new MissingResourceException(e.getMessage(), "File", e.getFile());
} catch (IOException e) {
    throw new ProcessingException("IO error occurred", e);
}
```

### 6. Use Java 10's `var` for Cleaner Rethrow

```
try {
    // code that throws multiple checked exceptions
} catch (final Exception e) {
    // do some handling
    throw e;  // Java 10+ allows this for multiple checked exceptions
}
```

## 7. Log Before Rethrowing

```
try {
    // code that may throw
} catch (SpecificException e) {
    logger.log(Level.WARNING, "Context about what failed", e);
    throw e;
}
```

## 8. Rethrow as Unchecked When Appropriate

```
try {
    // code that throws checked exception
} catch (CheckedException e) {
    throw new RuntimeException("Unexpected error", e);
}
```

## 9. Resource Cleanup

```
public void processResource() throws ApplicationException {
    Resource r = acquireResource();
    try {
        useResource(r);
    } catch (OperationException e) {
        try { r.cleanup(); } catch (CleanupException ce) { e.addSuppressed(ce); }
        throw new ApplicationException("Operation failed", e);
    } finally {
        try { r.close(); } catch (IOException e) { logger.warn("Close failed", e); }
    }
}
```

# 12.8. Chaining Exceptions

## 👉What is Exception Chaining?

✨The **Exception Chaining** is a technique where one exception is thrown while another exception is being handled.

✨It provides context about the original exception while throwing a new one.

**Syntax**: Chaining Exceptions

```
try {
    // Code that may throw an exception
} catch (OriginalException originalException) {
    throw new NewException("Message", originalException);
}
```

**Explain**:

✨The `NewException` is the new exception being thrown.

✨The `originalException` is the exception being wrapped, providing context.

# 👉When to Use Exception Chaining?

✨When you want to explain more about what went wrong.

✨When you want to keep details of the original error for debugging.

✨When you want to show both your own message and the original error.

✨When you want to pass an error up to another part of your program, but add extra information.

✨When you want to make it easier to find and fix problems by keeping the original error.

# 👉Best Practice for Exception Chaining

**1. Always Chain Exceptions Properly**: Basic chaining

```
try {
    // code that may throw IOException
} catch (IOException e) {
    throw new ServiceException("Failed to process request", e); // e is the cause
}
```

## 2. Preserve the Full Stack Trace:

✨ Good (preserves stack trace)

```
try {
    parseFile(configFile);
} catch (FileParseException e) {
    throw new ConfigurationException("Invalid config file: " + configFile.getName(), e);
}
```

✨ Bad (loses original stack trace)

```
try {
    parseFile(configFile);
} catch (FileParseException e) {
    // DON'T DO THIS - loses original stack trace
    throw new ConfigurationException("Invalid config file: " + e.getMessage());
}
```

### 3. Use Meaningful Exception Types:

```
public void processPayment(Payment payment) throws PaymentException {
    try {
        paymentGateway.process(payment);
    } catch (NetworkException e) {
        throw new PaymentException("Network error processing payment", e);
    } catch (DatabaseException e) {
        throw new PaymentException("Failed to record payment", e);
    }
}
```

### 4. Add Contextual Information:

```
try {
    db.executeUpdate(query);
} catch (SQLException e) {
    throw new DataAccessException(
        "Failed to execute query: " + query + ". Error code: " + e.getErrorCode(),
        e
    );
}
```

## 5. Implement Custom Exceptions Properly:

```
public class ApplicationException extends Exception {
    // Constructors that support chaining
    public ApplicationException(String message) {
        super(message);
    }
    public ApplicationException(String message, Throwable cause) {
        super(message, cause);
    }
    // Additional useful methods
    public String getRootCauseMessage() {
        Throwable root = this;
        while (root.getCause() != null) {
            root = root.getCause();
        }
        return root.getMessage();
    }
}
```

### 6. Log Appropriately:

```
try {
    processOrder(order);
} catch (ProcessingException e) {
    logger.error("Failed to process order {}: {}", order.getId(), e.getMessage(), e);
    throw new OrderException("Order processing failed", e);
}
```

### 7. Handle Multiple Levels of Chaining:

```
try {
    // business logic
} catch (BusinessException e) {
    if (e.getCause() instanceof IOException) {
        // Special handling for IO-related business failures
        handleIOFailure((IOException)e.getCause());
    }
    throw new PresentationLayerException("Operation failed", e);
}
```

## 8. Avoid Over-Chaining:

✨ Bad (too many layers)

```
try {
    // ...
} catch (LowLevelException e) {
    throw new MidLevelException(e); // Just adds noise
}
```

✨ Better

```
try {
    // ...
} catch (LowLevelException e) {
    throw new HighLevelException("Meaningful message", e); // Skip unnecessary layers
}
```

### 9. Use Java's Built-in Chaining Support:

```
// The cause can be accessed later
try {
    // ...
} catch (Exception e) {
    Exception wrapped = new ApplicationException("Context", e);
    System.out.println("Root cause: " + wrapped.getCause());
}
```

### 10. Document Chained Exceptions:

```
/**
 * Processes the data file
 * @throws DataProcessingException when processing fails, with these common causes:
 *   - IOException: When file cannot be read
 *   - ParseException: When file contains invalid data
 */
public void processData(File file) throws DataProcessingException {
    // ...
}
```

# 12.9. Defining Custom Exception Classes

# 👉Why Define Custom Exceptions?

✨Custom exceptions let you define your own error types for your program.

✨They make it easier to know what went wrong in your code.

✨You can add extra information to help understand the error.

✨They help make your code easier to read and fix.

✨You can handle different problems in different ways.

✨Useful for checking rules or special cases in your application.

✨You can organize errors better by creating groups of related exceptions.

✨They help you give clear error messages to users or developers.

**Example**: Creating a Custom Exception Class

```
public class MyCustomException extends Exception {
    public MyCustomException(String message) {
        super(message); // Call the parent class constructor
    }
}
```

**Explain**:

✨ The `MyCustomException` class extends the `Exception` class.

✨ It has a constructor that takes a message and passes it to the parent class constructor using `super(message)`.

**Example**: Throwing and Catching Custom Exceptions

```
public class CustomExceptionDemo {
    public static void main(String[] args) {
        try {
            throw new MyCustomException("This is a custom exception!");
        } catch (MyCustomException ex) {
            System.out.println("Caught: " + ex.getMessage());
        }
    }
}
```

**Explain**:

✨ The `main` method throws a `MyCustomException`.

✨ The `catch` block catches the custom exception and prints its message.

# 👉Best Practice for Defining Custom Exception

## 1. Use Meaningful Names:

```
// Custom exceptions
class BaseException extends Exception {
    public BaseException(String msg) { super(msg); }
}
class InputException extends BaseException {
    private Object value;
    public InputException(String msg, Object value) {
        super(msg + ". Value: " + value);
        this.value = value;
    }
    public Object getValue() { return value; }
}
```

## 2. Define Base and Derived Classes:

```
// Base and derived classes
abstract class Shape {
    private String name;
    public Shape(String name) throws InputException {
        if (name == null) throw new InputException("Name must not be null", name);
        this.name = name;
    }
    public String getName() { return name; }
    public abstract double area() throws BaseException;
}
```

### 3. Implement Specific Exceptions:

```
class Rectangle extends Shape {
    private double w, h;
    public Rectangle(double w, double h) throws InputException {
        super("Rectangle");
        if (w <= 0 || h <= 0)
            throw new InputException("Width/height must be positive", "w=" + w + ", h=" + h);
        this.w = w; this.h = h;
    }
    public double area() { return w * h; }
    public String toString() { return getName() + " w=" + w + ", h=" + h; }
}                                                                                    .
```

## 4. Use Custom Exceptions in Main Method:

```
public class Main {
    public static void main(String[] args) {
        try {
            Rectangle r1 = new Rectangle(5, 10);
            System.out.println(r1 + "\nArea: " + r1.area());
            Rectangle r2 = new Rectangle(-5, 10); // Throws InputException
        } catch (InputException e) {
            System.err.println("Input error: " + e.getMessage() + "\nInvalid: " + e.getValue());
        } catch (BaseException e) {
            System.err.println("Shape error: " + e.getMessage());
        }
        try {
            Shape s = new Shape(null) { public double area() { return 0; } };
        } catch (InputException e) {
            System.err.println("Failed to create shape: " + e.getMessage());
        } catch (BaseException e) {
            System.err.println("Unexpected error: " + e.getMessage());
        }
    }
}                                                                    .
```

# Phase-II:
# File I/O, Binary I/O, and Object I/O

# 12.10. The File Class

# 👉What is the File Class?

✨The `java.io.File` class represents a file or directory path in the file system. Commonly used methods in the `File` class

| Method | Description | Example Usage |
|---|---|---|
| `exists()` | Checks if the file or directory exists | `file.exists()` |
| `isFile()` | Checks if this path is a file | `file.isFile()` |
| `isDirectory()` | Checks if this path is a directory | `file.isDirectory()` |
| `getName()` | Returns the name of the file or directory | `file.getName()` |
| `getPath()` | Returns the path as a string | `file.getPath()` |
| `getAbsolutePath()` | Returns the absolute path | `file.getAbsolutePath()` |
| `length()` | Returns the file size in bytes | `file.length()` |
| `canRead()` | Checks if the file is readable | `file.canRead()` |
| `canWrite()` | Checks if the file is writable | `file.canWrite()` |

| Method | Description | Example Usage |
|---|---|---|
| `canExecute()` | Checks if the file is executable | `file.canExecute()` |
| `lastModified()` | Returns the last modified time (milliseconds) | `file.lastModified()` |
| `delete()` | Deletes the file or directory | `file.delete()` |
| `createNewFile()` | Creates a new empty file | `file.createNewFile()` |
| `mkdir()` | Creates a single directory | `file.mkdir()` |
| `mkdirs()` | Creates directories including any necessary parent dirs | `file.mkdirs()` |
| `list()` | Lists files/directories in a directory (as String array) | `file.list()` |
| `listFiles()` | Lists files/directories as `File` objects | `file.listFiles()` |
| `renameTo(File dest)` | Renames the file or directory | `file.renameTo(new File("newname.txt"))` |
| `setReadOnly()` | Sets the file to read-only | `file.setReadOnly()` |

**Example**: Creating File Objects

```
File file1 = new File("example.txt");
File file2 = new File("/path/to/directory");
File file3 = new File("C:\\path\\to\\file.txt");
```

**Explain**:

- ✨ `File` objects can be created using relative or absolute paths.
- ✨ Paths can be specified using forward slashes (/) or backslashes (\) depending on the operating system.

**Example:** Checking File Properties

```
File file = new File("example.txt");
if (file.exists()) {
    System.out.println("File size: " + file.length());
    System.out.println("Is directory? " + file.isDirectory());
}
```

**Output**:

```
File size: 1024
Is directory? false
```

**Example**: Checking File Properties

```
File file = new File("example.txt");
if (file.exists()) {
    System.out.println("File name: " + file.getName());
    System.out.println("Absolute path: " + file.getAbsolutePath());
    System.out.println("Is writable? " + file.canWrite());
} else {
    System.out.println("File does not exist.");
}
```

**Output**:

```
File name: example.txt
Absolute path: /path/to/example.txt
Is writable? true
```

# 👉Directory Operations

| Operation | Description | Example Code |
|---|---|---|
| Create Directory | Creates a new directory | `file.mkdir();` |
| Create Directories | Creates directory and parent directories | `file.mkdirs();` |
| List Files/Directories | Lists names of files/directories (String array) | `String[] files = file.list();` |
| List as File Objects | Lists files/directories as `File` objects | `File[] files = file.listFiles();` |
| Delete Directory/File | Deletes a directory or file | `file.delete();` |
| Rename Directory/File | Renames a directory or file | `file.renameTo(new File("newName"));` |
| Check if Directory/File | Checks if path is a directory or file | `file.isDirectory();` / `file.isFile();` |
| Get Parent Directory | Gets the parent directory path | `file.getParent();` |
| Get Absolute Path | Gets the absolute path of directory or file | `file.getAbsolutePath();` |

**Example:** Listing all files in a directory

```
File dir = new File("myfolder");
if (dir.isDirectory()) {
    File[] files = dir.listFiles();
    for (File f : files) {
        System.out.println(f.getName());
    }
}
```

**Output**:

```
file1.txt
file2.txt
subfolder
```

# 👉Common Pitfalls

- ✨Always check if a file exists before performing operations.
- ✨Use `try-catch` blocks to handle potential exceptions when working with files.
- ✨Use `File.separator` for platform-independent file paths.
- ✨Avoid hardcoding file paths; use relative paths or configuration files.
- ✨Be cautious with file permissions; ensure the program has the necessary rights to read/write files.
- ✨Use `File.createNewFile()` to avoid overwriting existing files unintentionally.
- ✨Always close file streams to prevent resource leaks.

# 12.11. File Input and Output (I/O)

# 👉Introduction to File I/O

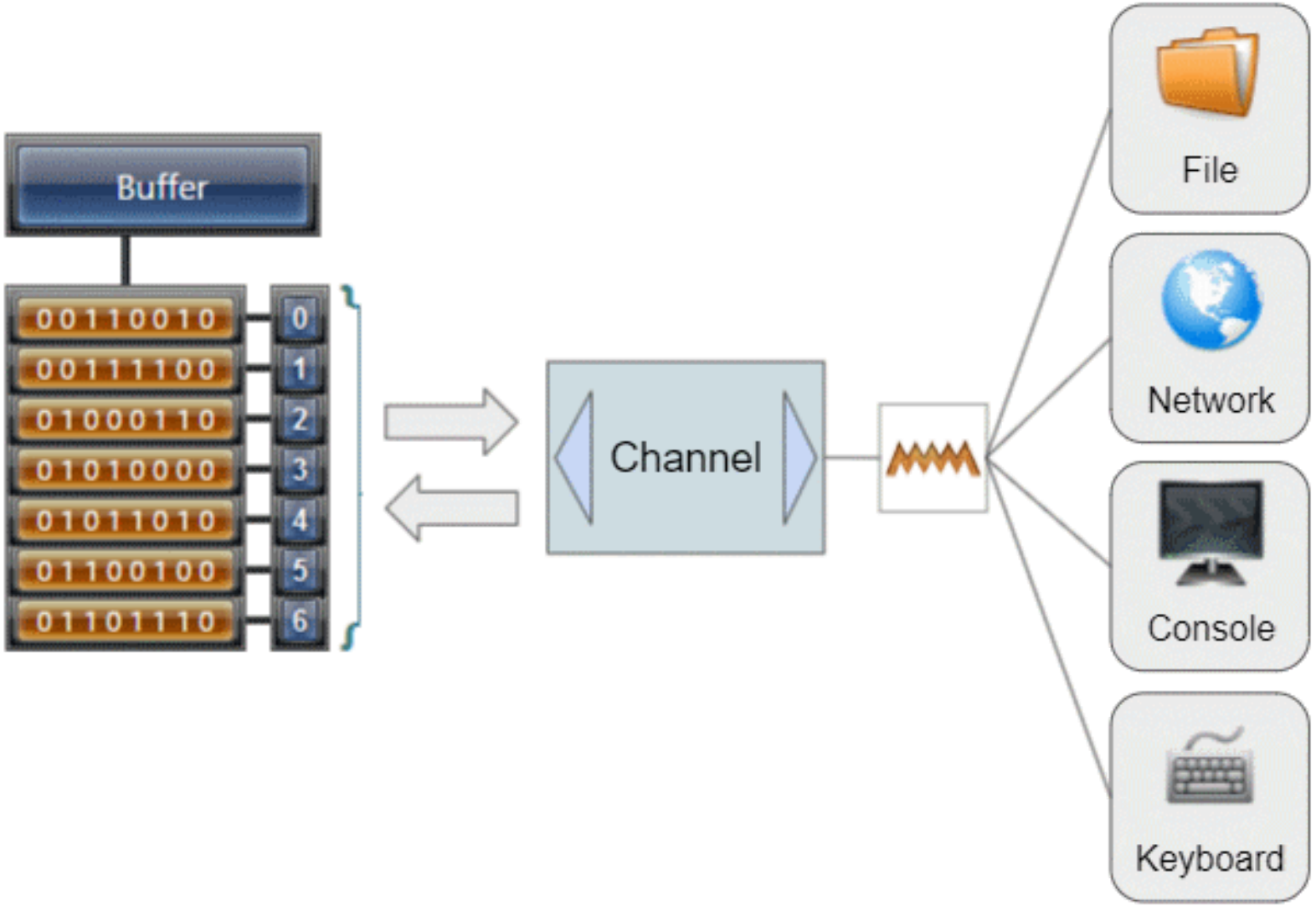
✨File I/O (Input/Output) is the process of reading from, writing to, saving, loading, sending, and receiving data to and from files.

✨Java has built-in classes for file I/O operations, such as:

- ✨ `FileReader` : Reads character files.
- ✨ `FileWriter` : Writes character files.
- ✨ `BufferedReader` : Reads text from a character-input stream, buffering characters for efficient reading.
- ✨ `BufferedWriter` : Writes text to a character-output stream, buffering characters for efficient writing.

**Note**: File I/O focuses on reading and writing type text files (txt, csv, json, ...); not for images, videos, or other binary data.

# 👉What is Buffered?



## **Example**: Reading Text Files

```
import java.io.*;
public class ReadFileExample {
    public static void main(String[] args) {
        try (BufferedReader reader = new BufferedReader(new FileReader("example.txt"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

**Explain**:

- ✨ The `BufferedReader` class is used to read text from a file efficiently.
- ✨ The `FileReader` class is used to read the contents of a file.
- ✨ The `try-with-resources` statement ensures that the file is closed automatically after use, preventing resource leaks.
- ✨ The `IOException` is caught to handle any errors that may occur during file operations.

## **Example**: Writing Text Files

```
import java.io.*;
public class WriteFileExample {
    public static void main(String[] args) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt"))) {
            writer.write("Hello, World!");
            writer.newLine();
            writer.write("This is a text file.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

**Explain**:

- ✨ The `BufferedWriter` class is used to write text to a file efficiently.
- ✨ The `try-with-resources` statement ensures that the file is closed automatically after use, preventing resource leaks.
- ✨ The `IOException` is caught to handle any errors that may occur during file operations.

## **Example**: Handling File Exceptions

```
import java.io.*;
public class FileExceptionExample {
    public static void main(String[] args) {
        File file = new File("nonexistent.txt");
        try {
            if (!file.exists()) {
                throw new IOException("File does not exist: " + file.getAbsolutePath());
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

**Explain**:

✨ The `File` class is used to represent a file or directory path.

✨ The `exists()` method checks if the file exists.

✨ If the file does not exist, an `IOException` is thrown with a custom message.

## **Example**: Closing Files and Resource Management

```
import java.io.*;
public class CloseFileExample {
    public static void main(String[] args) {
        BufferedReader reader = null;
        try {
            reader = new BufferedReader(new FileReader("example.txt"));
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            if (reader != null) {
                try {
                    reader.close(); // Ensure the file is closed
                } catch (IOException e) {
                    e.printStackTrace();
}}}}}                                                                    .
```

**Explain**:

- ✨ The `BufferedReader` is used to read text from a file.
- ✨ The `try-catch-finally` structure ensures that the file is closed properly, even if an exception occurs.
- ✨ The `finally` block is used to close the `BufferedReader`, ensuring that resources are released.

## **Example**: Copying Files

```
import java.io.*;
public class CopyFileExample {
    public static void main(String[] args) {
        try (BufferedReader in = new BufferedReader(new FileReader("source.txt"));
             BufferedWriter out = new BufferedWriter(new FileWriter("destination.txt"))) {
            for (String line; (line = in.readLine()) != null; )
                out.write(line + System.lineSeparator());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

**Explain**:

✨ This example demonstrates how to copy the contents of one text file to another.

✨ The `BufferedReader` reads lines from the source file, and the `BufferedWriter` writes them to the destination file.

✨ The `try-with-resources` statement ensures that both readers and writers are closed automatically after use, preventing resource leaks.

✨ The `IOException` is caught to handle any errors that may occur during file operations.

# 👉Common Pitfalls

✨Always close files when you are done. Use `try-with-resources` to do this automatically.

✨Catch and handle exceptions so your program does not crash.

✨Use the right text encoding (like UTF-8) when reading or writing text files.

✨Use `BufferedReader` and `BufferedWriter` for faster reading and writing.

✨Do not hardcode file paths; use relative paths or settings.

✨Check if a file exists before you try to read or write it.

**UTF-8**: Universal character encoding that supports all characters in the Unicode standard.

# 12.12. Use Case: Reading Data from the Web

# 👉Reading Data from the Web

✨Java can read data from websites and online files.

✨Use the `URL` class for web addresses (like "https://www.example.com").

✨Use the `URLConnection` class to connect and get data.

✨Read website text with input streams, similar to reading a file.

✨Always handle errors, since network issues can happen.

✨Check if the web address is correct and set timeouts to avoid waiting forever.

## **Example**: The `URL` and `URLConnection` Classes

```
import java.net.*;
import java.io.*;

public class WebDataExample {
    public static void main(String[] args) throws Exception {
        URL url = new URL("https://www.example.com");
        try (BufferedReader in = new BufferedReader(
                new InputStreamReader(url.openStream()))) {
            String line;
            while ((line = in.readLine()) != null)
                System.out.println(line);
        }
    }
}
```

**Explain**:

- ✨ The `URL` class is used to create a URL object representing the web address.
- ✨ The `URLConnection` class is used to open a connection to the URL.
- ✨ The `InputStreamReader` and `BufferedReader` classes are used to read the content of the web page line by line.

# 👉Best Practicle for Handling Exceptions in Web I/O

## 1. Resource Cleanup:

```
// Always close resources in finally block
try (InputStream is = url.openStream()) {
    // use the stream
} catch (IOException e) {
    // handle exception
}
```

## 2. HTTP Client Libraries:

```
// Using HttpClient (Java 11+)
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://example.com"))
    .build();

try {
    HttpResponse<String> response =
        client.send(request, HttpResponse.BodyHandlers.ofString());
    // Process response
} catch (IOException | InterruptedException e) {
    // Handle exceptions
}
```

### 3. Retry Mechanism:

```
public static String fetchWithRetry(String url, int maxRetries) throws WebIOException {
    int retryCount = 0;
    while (true) {
        try {
            return fetchUrlContent(url);
        } catch (WebIOException e) {
            if (retryCount >= maxRetries ||
                !(e.getCause() instanceof SocketTimeoutException)) {
                throw e;
            }
            retryCount++;
            try {
                Thread.sleep(1000 * retryCount);
            } catch (InterruptedException ie) {
                Thread.currentThread().interrupt();
                throw new WebIOException("Interrupted during retry", ie);
}}}}
```

## 4. Handling Different Content Types:

```
public static <T> T fetchJson(String url, Class<T> responseType) throws WebIOException {
    try {
        String json = fetchUrlContent(url);
        ObjectMapper mapper = new ObjectMapper();
        return mapper.readValue(json, responseType);
    } catch (JsonProcessingException e) {
        throw new WebIOException("Failed to parse JSON response", e);
    }
}
```

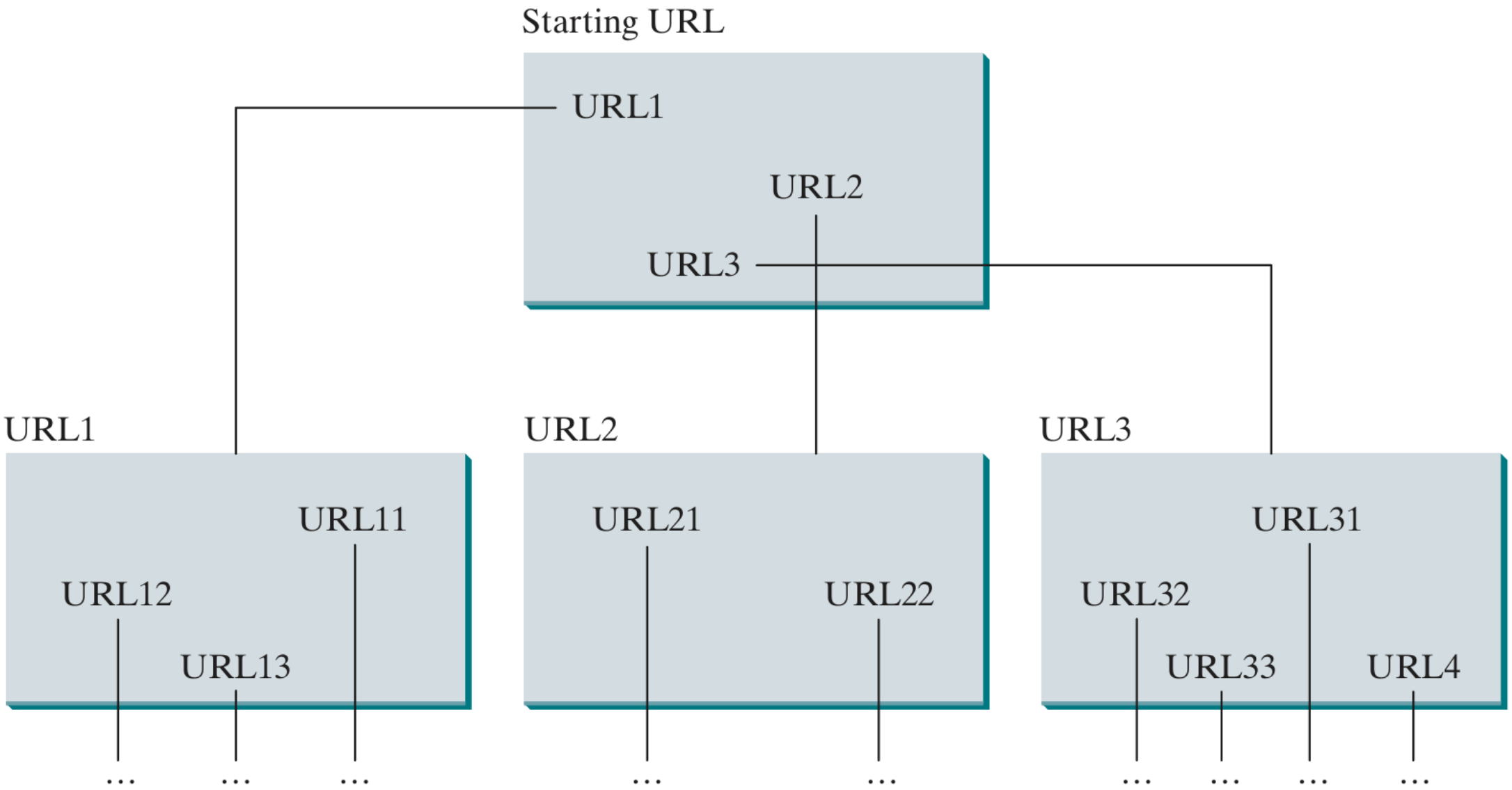
## 5. Async Web I/O:

```
public static CompletableFuture<String> fetchAsync(String url) {
    HttpClient client = HttpClient.newHttpClient();
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(url))
        .build();

    return client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
        .thenApply(HttpResponse::body)
        .exceptionally(ex -> {
            System.err.println("Async request failed: " + ex.getMessage());
            return null;
        });
}
```

# 12.13. Case Study: Web Crawler

Practice.


Starting URL
URL1
URL2
URL3
URL1
URL11
URL12
URL13
URL2
URL21
URL22
URL3
URL31
URL32
URL33
URL4

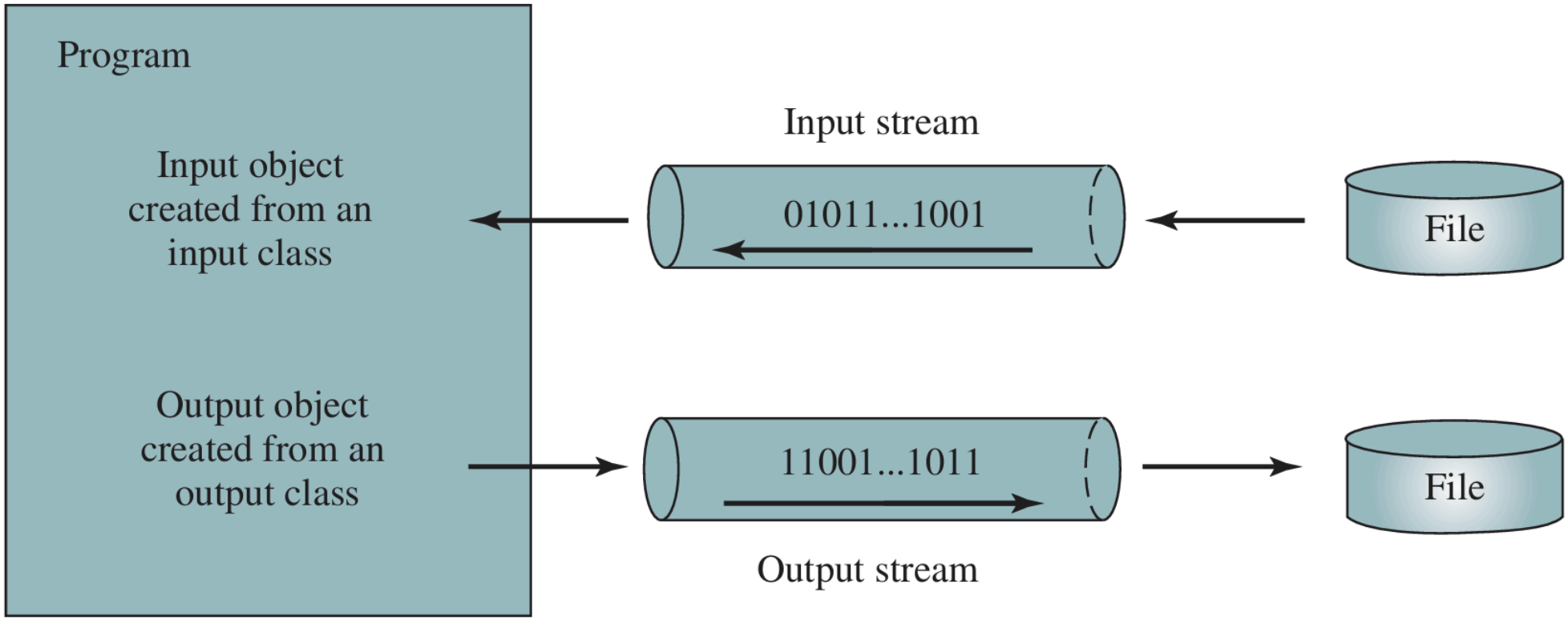
# 17.1. Introduction to Binary I/O

# 👉Introduction to Binary I/O

✨The **Binary I/O** is the process of reading and writing data in binary format.

✨Data can be sent and received in binary stream format.

✨**Stream** is a sequence of data elements made available over time.



# 👉Efficiency of Binary Files

✨Binary files are more efficient to process than text files because they store data in a compact format.

✨They do not require character encoding or decoding, making them faster to read and write.

✨Text files use character-encoding schemes like ASCII or Unicode.

✨**Note**: Binary files store data in raw binary format.

# 17.2. How Is Text I/O Handled in Java?

# 👉Introduction to Text I/O in Java

✨**Text I/O** is the process of reading and writing text data in Java.

✨Text data is read using the `Scanner` class.

```
Scanner input = new Scanner(new File("temp.txt"));
while (input.hasNextLine()) {
    String line = input.nextLine();
    System.out.println(line);
}
```

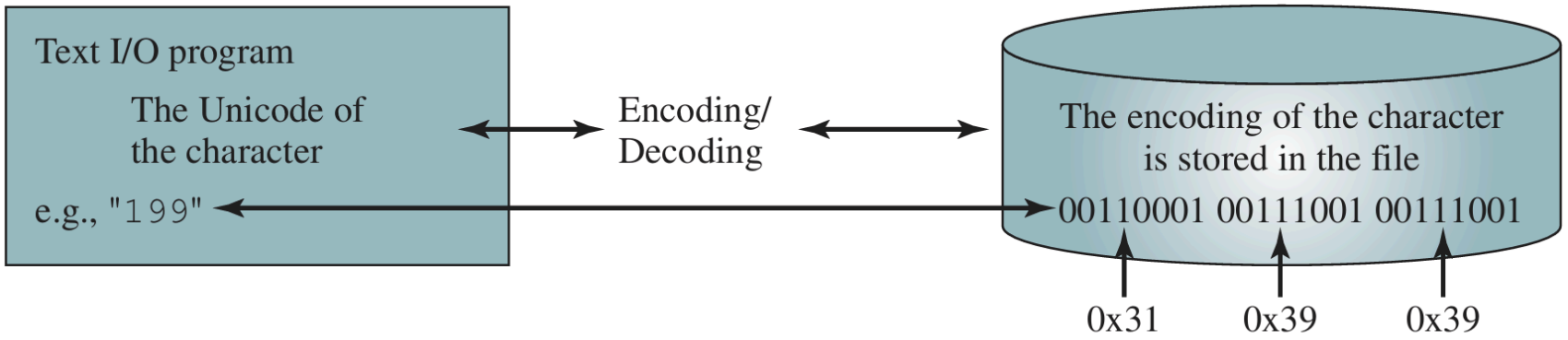
✨Text data is written using the `PrintWriter` class.

```
PrintWriter output = new PrintWriter("temp.txt");
output.println("Hello, World!");
output.close();
```

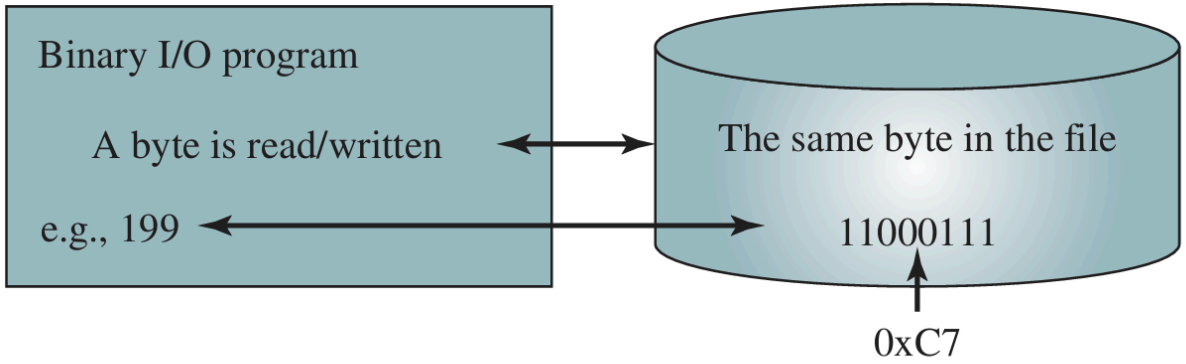
# 17.3. Text I/O vs. Binary I/O

# 👉Text File vs. Binary File

| Feature | Text File | Binary File |
|---|---|---|
| **Content Format** | Stores data as readable characters (ASCII/Unicode) | Stores data as raw bytes (not human-readable) |
| **Usage** | For text data: documents, source code, logs | For images, audio, video, executables, serialized objects |
| **Encoding** | Requires character encoding/decoding | No encoding/decoding; data is written as-is |
| **Readability** | Can be opened and read by text editors | Not readable by text editors |
| **Efficiency** | Less efficient for large/complex data | More efficient for storage and processing |
| **Portability** | May have issues with encoding differences (UTF-8, etc.) | Portable if read/written by compatible programs |
| **Examples** | `.txt`, `.csv`, `.xml`, `.json` | `.jpg`, `.mp3`, `.exe`, `.dat`, `.class` |

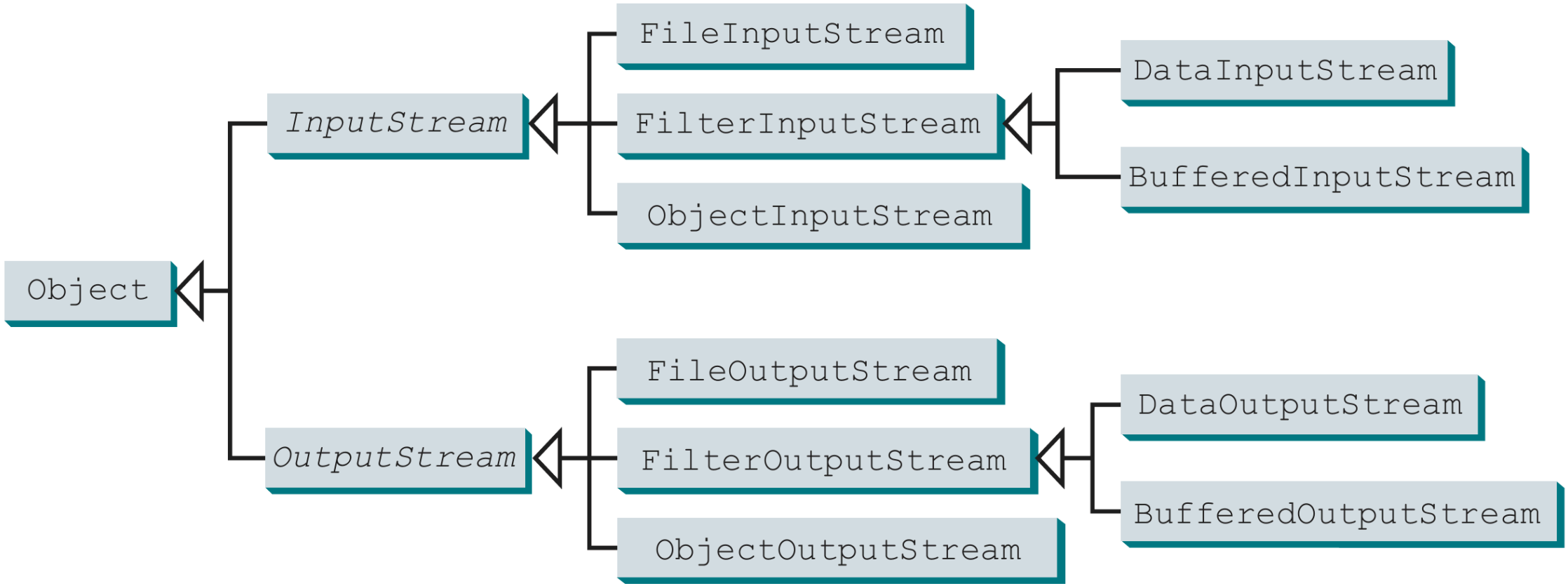

Text I/O program
The Unicode of the character
e.g., "199"
Encoding/ Decoding
The encoding of the character is stored in the file
00110001 00111001 00111001
0x31
0x39
0x39

(a)


Binary I/O program
A byte is read/written
e.g., 199
The same byte in the file
11000111
0xC7

(b)

# 17.4. Binary I/O Classes


Object
InputStream
FileInputStream
FilterInputStream
DataInputStream
BufferedInputStream
ObjectInputStream
OutputStream
FileOutputStream
FilterOutputStream
DataOutputStream
BufferedOutputStream
ObjectOutputStream

# 👉Introduction to Binary I/O Classes

- ✨**InputStream**: Used for reading binary data.
  - ✨**FileInputStream**: Reads bytes from a file.
  - ✨**FilterInputStream**: Wraps another input stream to provide additional functionality.
    - ✨**DataInputStream**: Reads primitive data types (int, float, etc.) and strings.
    - ✨**BufferedInputStream**: Reads bytes from an input stream, buffering for efficiency.
  - ✨**ObjectInputStream**: Reads objects from a binary stream, allowing for serialization and deserialization.

- ✨ **OutputStream**: Used for writing binary data.
  - ✨ **FileOutputStream**: Writes bytes to a file.
  - ✨ **FilterOutputStream**: Wraps another output stream to provide additional functionality.
    - ✨ **DataOutputStream**: Writes primitive data types (int, float, etc.) and strings.
    - ✨ **BufferedOutputStream**: Writes bytes to an output stream, buffering for efficiency.
  - ✨ **ObjectOutputStream**: Writes objects to a binary stream, allowing for serialization and deserialization.

## 👉 `FileInputStream` and `FileOutputStream`

✨ **FileInputStream/FileOutputStream** are used for reading and writing binary data to files. They read and write bytes directly, making them suitable for binary files.

**Example**: Reading from a File

```
import java.io.*;
public class FileIOExample {
    public static void main(String[] args) {
        try (FileInputStream input = new FileInputStream("input.dat")) {
            int data;
            while ((data = input.read()) != -1) {
                System.out.print((char) data);
            }
        } catch (IOException e) {
            e.printStackTrace();
}}}
```

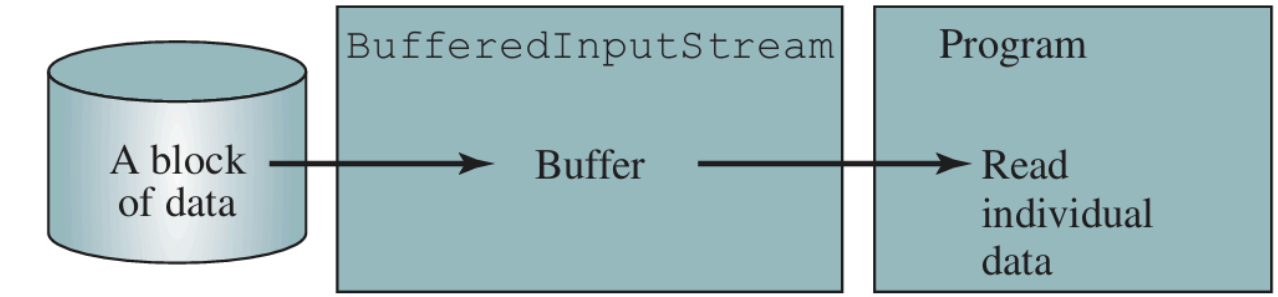
## 👉 `BufferedInputStream` and `BufferedOutputStream`

✨ **Buffered** is a technique to improve I/O performance by reducing the number of read/write operations. It uses a buffer (an array of bytes) to store data temporarily before reading or writing.

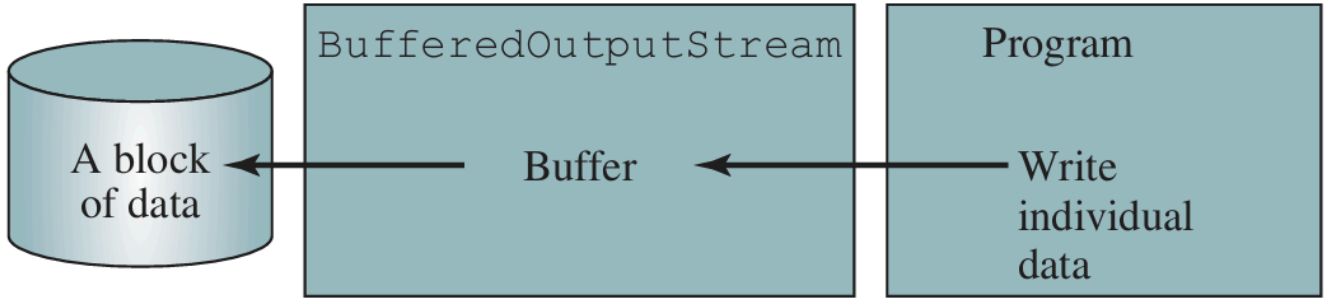
**Example**: Using Buffered Streams

```
import java.io.*;
public class BufferedIOExample {
    public static void main(String[] args) throws IOException {
        try (var in = new BufferedInputStream(new FileInputStream("input.dat"));
             var out = new BufferedOutputStream(new FileOutputStream("output.dat"))) {
            for (int b; (b = in.read()) != -1; ) out.write(b);
        }
    }
}
```

**Note**: `var` is used for type inference in Java 10 and later.


BufferedInputStream
Program
A block of data
Buffer
Read individual data

(a)


BufferedOutputStream
Program
A block of data
Buffer
Write individual data

(b)

## 👉 `DataInputStream` and `DataOutputStream`

✨ **DataInputStream** and **DataOutputStream** are used for reading and writing primitive data types (int, float, double, etc.) and strings in a binary format.

**Example**: Using Data Streams

```
import java.io.*;
public class DataStreamExample {
    public static void main(String[] args) throws IOException {
        try (var out = new DataOutputStream(new FileOutputStream("data.dat"))) {
            out.writeUTF("John"); out.writeDouble(85.5);
        }
        try (var in = new DataInputStream(new FileInputStream("data.dat"))) {
            System.out.println("Name: " + in.readUTF() + ", Score: " + in.readDouble());
        }
    }
}
```

**Note**: `var` is used for type inference in Java 10 and later.

# 17.5. Case Study: Copying Files

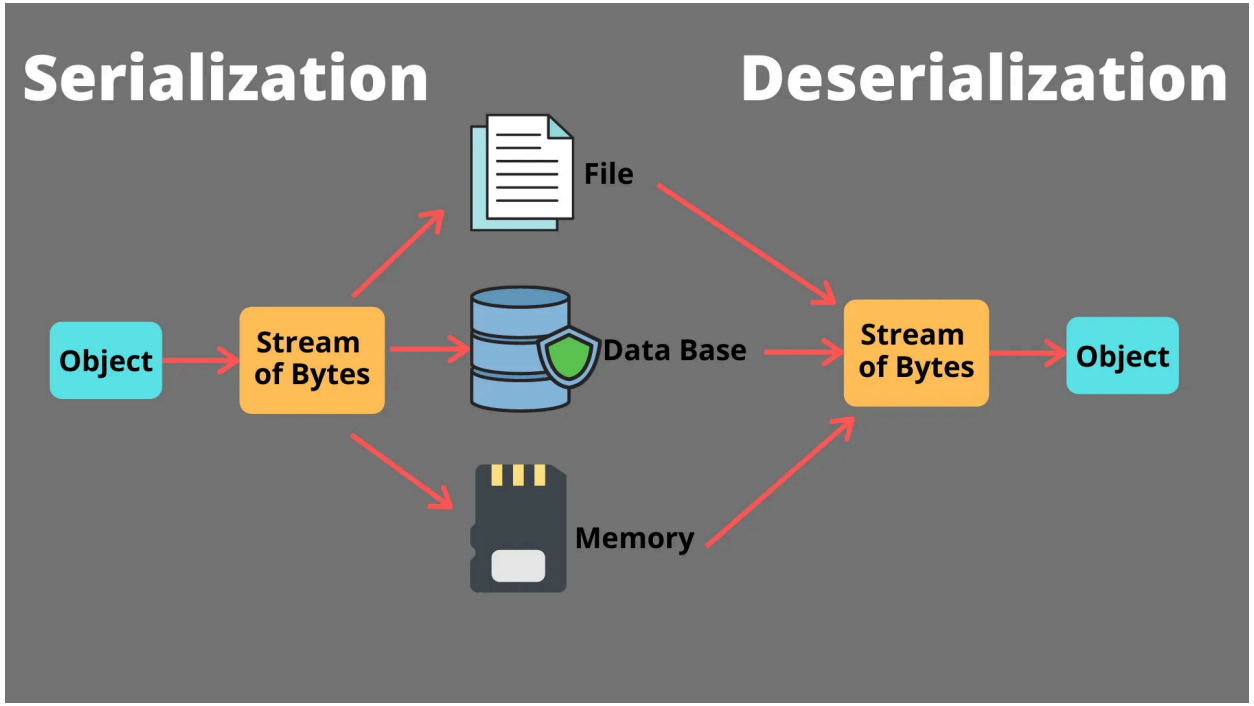
Practice.

# 17.6. Object I/O

# 👉Introduction to Object I/O

✨Did you know that objects in Java can be read from and written to files, sent and received over networks, as well as stored and retrieved for later use?

✨In Java, objects can be serialized and deserialized, allowing them to be converted into a format that can be easily stored or transmitted.



# 👉Object I/O in Java

✨To read and write objects, Java provides the `ObjectInputStream` and `ObjectOutputStream` classes from the `java.io` package.

**Example**: ObjectOutputStream

```
ObjectOutputStream output = new ObjectOutputStream(new FileOutputStream("object.dat"));
output.writeObject(new Date());
output.close();
```

**Example**: ObjectInputStream

```
ObjectInputStream input = new ObjectInputStream(new FileInputStream("object.dat"));   .
Date date = (Date) input.readObject();
input.close();
```

**Explain**:

- ✨ `ObjectOutputStream` is used to write objects to a file.
- ✨ `ObjectInputStream` is used to read objects from a file.
- ✨ The `writeObject()` method serializes the object, and the `readObject()` method deserializes it.
- ✨ The object must implement the `Serializable` interface to be written to a stream.

## 👉Serializable Interface

✨To save objects to a file or send them over a network, the class must use `implements Serializable`. This is a special marker interface (it has no methods) that tells Java the object can be serialized.

**Example**: Serializable Interface

```
import java.io.Serializable;
public class Student implements Serializable {
    private String name;
    private double score;
    public Student(String name, double score) {
        this.name = name;
        this.score = score;
    }
    // Getters and toString() method
}
```

**Example**: Object I/O

✨ Writing objects:

```
output.writeUTF("John");
output.writeDouble(85.5);
output.writeObject(new Date());
```

✨ Reading objects:

```
String name = input.readUTF();
double score = input.readDouble();
Date date = (Date) input.readObject();
```

# 👉Handling Non-Serializable Fields

✨To handle non-serializable fields, mark them as `transient`.

**Syntax**:

```
import java.io.Serializable;
public class MyClass implements Serializable {
    private String name;
    private transient NonSerializableType nonSerializableField;

    // Constructor, getters, setters, etc.
}
```

**Explain**: `transient` keyword indicates that the field should not be serialized.

# 👉Serializing Arrays

✨**Arrays** are serializable if all elements are serializable.

**Example**: Writing an Array

```
int[] numbers = {1, 2, 3, 4, 5};
output.writeObject(numbers);
```

**Example**: Reading an Array

```
int[] numbers = (int[]) input.readObject();
System.out.println(Arrays.toString(numbers));
```

**Explain**:

✨ Arrays can be serialized just like objects.

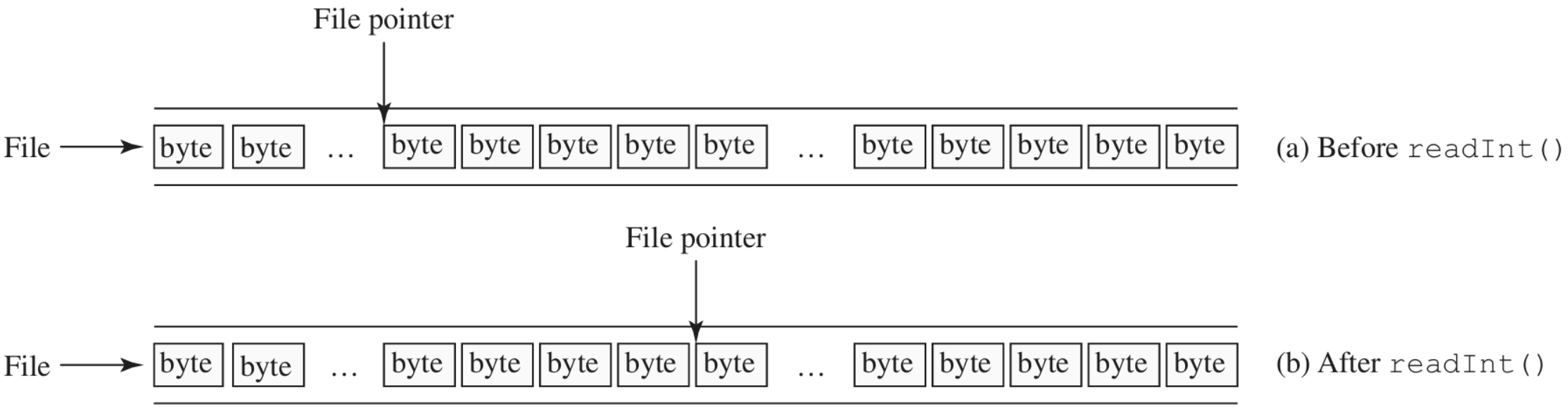
✨ The `writeObject()` method can be used to write an array, and the `readObject()` method can be used to read it back.

# 17.7. Random-Access Files

# 👉Introduction to Random-Access Files

✨The **Random-Access File** allows you to read and write data at any position in a file.

✨This means you can jump to a specific part of the file without reading everything before it.



## 👉RandomAccessFile Class

✨The `RandomAccessFile` class is used for reading and writing files in a random-access manner.

**Syntax**:

```
RandomAccessFile raf = new RandomAccessFile("filename", "mode");
```

**Note**:

✨The `filename` is the name of the file to be accessed.

✨The `mode` specifies how the file will be accessed:

- ✨ `"r"` : Read-only mode.
- ✨ `"rw"` : Read and write mode.
- ✨ `"rws"` : Read/write mode with synchronization.

## 👉File Pointer

✨The **File Pointer** indicates the current position in the file where the next read or write operation will occur.

✨You can move the file pointer to a specific position using the `seek(long pos)` method.

**Example**: Moving the File Pointer

```
import java.io.RandomAccessFile;
public class RandomAccessFileExample {
    public static void main(String[] args) throws Exception {
        RandomAccessFile raf = new RandomAccessFile("example.dat", "rw");
        raf.writeInt(42); // Write an integer at the beginning
        raf.seek(0); // Move to the beginning
        int value = raf.readInt(); // Read the integer
        System.out.println("Value: " + value);
        raf.close();
    }
}
```

# 👉Reading and Writing Data

**Example**: Writing and Reading Data Using RandomAccessFile

```
import java.io.RandomAccessFile;
public class RandomAccessFileExample {
    public static void main(String[] args) throws Exception {
        RandomAccessFile raf = new RandomAccessFile("data.dat", "rw");
        raf.writeInt(100); // Write an integer
        raf.writeDouble(3.14); // Write a double
        raf.seek(0); // Move to the beginning
        int intValue = raf.readInt(); // Read the integer
        double doubleValue = raf.readDouble(); // Read the double
        System.out.println("Integer: " + intValue + ", Double: " + doubleValue);
        raf.close();
    }
}
```

# End of OOP